

Effect of Graph Partitioning on Link Prediction with Bilinear, Hadamard, and Random Forest Decoders over Static Node2Vec Embeddings

Stefan Nožinić

Abstract

This paper studies how graph partitioning affects link prediction when node embeddings are computed independently on each partition with static node2vec and merged afterwards. Unlike graph reconstruction, which asks how well embeddings recover the graph they were trained on, link prediction asks how well they generalize to held-out edges, and the two questions turn out to have opposite answers under partitioning. Using four real-world graphs (CITESEER, AstroPh, Cit-HepPh, AS-Oregon), partition counts $P \in \{1, 2, 4, 8\}$, and 10 independent runs per configuration (480 runs), we compare three link-prediction decoders on the merged embeddings: a Hadamard-product logistic decoder, a bilinear decoder $u^\top W v$, and a random forest on concatenated embeddings. Decoders are evaluated with a ranking protocol (Hits@k and mean reciprocal rank, MRR) in which each held-out edge is ranked against 500 negative candidates, and configurations are compared with Mann–Whitney tests and Cohen’s d . Partitioning strongly degrades the neural decoders: from $P = 1$ to $P = 8$ the MRR of the bilinear decoder falls by 39–60% (significantly at 11 of 12 partition-count steps) and that of the Hadamard decoder by 8–39%, mostly in the first step, whereas the MRR of the random forest is far less sensitive, changing between +4% and -26% (significantly at 5 of 12 steps, all on CITESEER and Cit-HepPh except one improvement on AS-Oregon). Consequently the decoders are ordered random forest > bilinear > Hadamard in most configurations, with the bilinear decoder best only on unpartitioned dense graphs. In a preliminary single-dataset comparison, label propagation (LPA) partitioning gave better link prediction than the Lancichinetti-Fortunato-Kertesz method (LFM) at every partition count tested. Finally, the reconstruction F1 score of the same embeddings increases with the partition count on three of the four datasets while neural-decoder link prediction deteriorates, so reconstruction quality is not a reliable proxy for downstream link-prediction quality in a partitioned embedding pipeline.

Introduction

A graph is a mathematical structure consisting of vertices (or nodes) connected by edges. Graphs are widely used to model relationships and interactions in various domains (Van Der Hofstad 2024), such as social networks (Leskovec et al. 2010b) (Backstrom et al. 2006) (Rozemberczki and Sarkar 2021), collaboration networks (Savić et al. 2017), terrorist networks (Krebs 2002) and blog citation networks (Adamic and Glance 2005). Graph vertex embeddings represent vertices as low-dimensional vectors, enabling machine learning tasks such as vertex classification, community detection, and link prediction (Leskovec et al. 2010a). Among these downstream tasks, link prediction is of particular practical importance: it underlies friend and content recommendation, knowledge-graph completion (Bordes et al. 2013), and the inference of missing or future edges in evolving networks.

State-of-the-art embedding methods such as node2vec (Grover and Leskovec 2016) compute embeddings from a single, monolithic training pass over the whole graph. When the graph is too large to embed on one machine, a common strategy is to partition the graph, embed each partition independently, and combine the resulting per-partition embeddings for downstream use (Fang et al. 2023) (Lombardo and Poggi 2019). Link prediction poses practically relevant question: how well do the embeddings generalize to edges that were *not* observed during training? Because independently embedded partitions have no shared frame of reference — each partition’s embedding space is the arbitrary output of a separate skip-gram training run (Church 2017) — a decoder or classifier that combines embeddings from different partitions must cope with per-partition embedding spaces that are not directly comparable to one another. This raises questions that do not arise in the single-partition setting: which partitioning strategy best preserves the community structure that link prediction relies on, which decoder architecture best tolerates misaligned per-partition latent spaces, and whether a downstream classifier can be made robust to this misalignment at all.

Problem Formulation

Given a static graph partitioned into P partitions, each embedded independently with static node2vec, we want to know how

- the number of partitions P , and the partitioning strategy (LFM vs. LPA), and
- the link-prediction decoder (bilinear, Hadamard, or random forest)

affect link-prediction quality, and how that quality relates to the graph-reconstruction quality of the same embeddings.

Related Work

Graph vertex embedding is a well-studied area, with various methods proposed to generate low-dimensional representations of vertices in a graph. Node2Vec (Grover and Leskovec 2016) is the state-of-the-art method used throughout this paper; it generates embeddings via biased random walks and has been shown to be effective at capturing community structure. As its improvement, DistGER (Fang et al. 2023) is a distributed graph embedding method that extends Node2Vec by leveraging distributed computing to handle large graphs, and (Lombardo and Poggi 2019) proposes an actor-model-based distributed framework for the same purpose. The common ground for these methods is that they generate walks which are later used to train a Word2Vec model (Church 2017) commonly used for generating embeddings in natural language processing tasks.

Link prediction from vertex embeddings is commonly formulated as a binary classification problem over pairs of vertices, using a decoder that maps a pair of embeddings to a link probability. The original node2vec paper (Grover and Leskovec 2016) popularized the use of simple binary operators — including the Hadamard product, which is one of the decoders compared in this paper — to combine two node embeddings into a single edge feature vector before classification. Leskovec et al. (Leskovec et al. 2010a) study link sign and formation prediction more broadly, motivating the classification-based framing of link prediction adopted here. Beyond the Hadamard product, a learned bilinear form over the two embeddings is a common, more expressive scoring function, and non-neural classifiers such as random forests (Breiman 2001) can be applied directly to the concatenated embeddings.

Evaluation protocol matters as much as the decoder. Li et al. (Li et al. 2023) point out that link-prediction results are often not comparable because of inconsistent evaluation settings, and propose the HeaRT benchmark, which ranks every positive test edge against a set of 500 negative candidates and reports the mean reciprocal rank (MRR) and Hits@k. The ranking protocol used in this paper follows this per-positive design, but draws the candidates uniformly at random instead of selecting hard negatives with heuristics.

Partitioning a graph while preserving the community structure it relies on is itself a well-studied problem. Community-detection-based partitioners are the natural choice when partitioning is intended to preserve locality for downstream embedding: the label propagation algorithm (LPA) (Raghavan et al. 2007) assigns each vertex the most frequent label among its neighbors, iterating until labels stabilize into communities, while the Lancichinetti-Fortunato-Kertesz (LFM) method (Lancichinetti et al. 2009) detects (possibly overlapping) communities by greedily optimizing a local fitness function around a seed vertex. Both methods have been used as static graph partitioners in this line of work, and this paper compares their impact on downstream link-prediction quality, complementing the classical static partitioning literature surveyed there (Benlic and Hao 2010)

(Sanders and Schulz 2012) (Sanders and Schulz 2011) (Romero Ruiz and Segura 2018) (Çatalyürek et al. 2023).

Distributed embedding systems that partition a graph before embedding face a fundamental representation problem once embeddings must be combined across partitions: because each partition is embedded independently, there is no guarantee that geometrically similar vectors in two different partitions’ embedding spaces represent structurally similar vertices. This paper studies the practical consequences of that misalignment for link prediction.

Contributions

This paper makes the following contributions:

- We measure, on four real-world graphs with 10 independent runs per configuration, how the partition count affects link prediction over independently computed static node2vec embeddings for three decoders (bilinear, Hadamard, random forest), using a per-positive ranking protocol (Hits@k, MRR).
- We show that the neural decoders lose a large part of their ranking quality when the graph is partitioned, whereas a random forest on concatenated embeddings is far less sensitive to the partition count, and we derive a decoder ordering (random forest > bilinear > Hadamard) together with the conditions in which it changes.
- We show that graph-reconstruction F1 and neural-decoder link-prediction quality move in opposite directions as the partition count grows, so reconstruction quality is not a reliable proxy for downstream link-prediction quality, and we compare LFM and LPA partitioning in a preliminary experiment.

Paper Organization

The rest of the paper is organized as follows: first, the system overview is presented, then graph partitioning, the embedding model, and link-prediction decoders and classifiers are described. Next, benchmarks and evaluation metrics are explained in detail. Finally, results and discussion are presented, followed by the conclusion and references.

Methods

System overview

This paper considers a fully known static graph $G = (V, E)$. The pipeline has five stages: (1) a fraction of the edges is held out as positive test pairs, and an equal number of non-adjacent vertex pairs is drawn as negative test pairs; (2) the

remaining training graph G_{train} is partitioned once into P partitions; (3) each partition’s induced subgraph is embedded independently with static node2vec; (4) the per-partition embeddings are merged into one embedding per vertex, on which a link-prediction decoder is trained; and (5) the decoder is evaluated on the held-out pairs. Because each partition is embedded in isolation from the others, the resulting per-partition embedding spaces are not trained to be mutually consistent: a large dot product between two vectors in partition p_i ’s embedding space carries no guaranteed relationship to a large dot product in partition p_j ’s embedding space. This is the central complication that distinguishes distributed link prediction from the single-partition case, and every design choice examined in this paper — partitioning strategy and decoder — is, in one way or another, a response to it.

A candidate pair (u, v) may span two different partitions. For a pair whose vertices lie in the same partition, both embeddings come from the same, internally consistent embedding space. For a pair whose vertices lie in different partitions, the decoder must combine embeddings from two independently trained spaces, and it is this cross-partition case that is expected to drive most of the quality loss observed in the Results section.

Graph partitioning

Both partitioners are community-detection-based and follow the same two-step scheme: detect communities in G_{train} , then pack the communities into exactly P partitions with a greedy, size-balancing bin-packing heuristic (communities sorted by size, each placed into the currently smallest partition) (Gupta and Ho 1999). Vertices of G_{train} therefore stay together with their community, and the partition sizes are kept approximately equal.

LPA (Label Propagation Algorithm) (Raghavan et al. 2007): every vertex is initialized with a unique label $\ell(v) = v$. In each sweep, vertices are visited in random order and each vertex adopts the label held by the plurality of its neighbors, with ties broken uniformly at random,

$$\ell(v) \leftarrow \arg \max_{\ell} |\{w \in N(v) : \ell(w) = \ell\}|,$$

until no label changes. Vertices sharing a final label form a community. The asynchronous variant is used, in which updated labels are visible immediately within a sweep. LPA has no tunable parameters and produces communities of very different sizes, which the bin-packing step then groups into P partitions. Communities are disjoint, so every vertex belongs to exactly one partition.

LFM (Lancichinetti-Fortunato-Kertesz method) (Lancichinetti et al. 2009): a community C is grown greedily from a randomly chosen seed vertex that is not yet covered. Community quality is measured by the fitness

$$f(C) = \frac{k_{\text{in}}(C)}{(k_{\text{in}}(C) + k_{\text{out}}(C))^\alpha}$$

where $k_{\text{in}}(C)$ and $k_{\text{out}}(C)$ are the total internal and external degrees of C and α is a resolution parameter. At each step the neighboring vertex with the largest positive fitness gain is added, vertices whose removal increases fitness are dropped, and growth stops when no neighbor improves fitness. Communities produced this way may overlap. A modified variant is used here, which (i) repeats seeding until the fraction of uncovered vertices falls to a threshold τ , (ii) caps the number of communities at $10P$ to guarantee termination when growth does not converge, (iii) pads with empty communities if fewer than P are found, and (iv) assigns any vertex still uncovered to a uniformly random community. Parameters are $\alpha = 1$ and $\tau = 0$ (every vertex must be covered). Because LFM communities can overlap, a vertex may end up in more than one partition after bin packing; this is handled in the embedding-merging step below.

Both methods partition the graph based on its community structure. Because the full training graph is known in advance, both can consider the entire neighborhood of every vertex.

Embedding model

Each partition’s induced subgraph $G_{\text{train}}[p_i]$ is embedded independently using static node2vec (Grover and Leskovec 2016). Node2vec generates r second-order random walks of length l from every vertex. For a walk that has just moved from t to x , the unnormalized probability of stepping next to a neighbor y of x is

$$\pi(x, y \mid t) = \begin{cases} 1/p & \text{if } y = t, \\ 1 & \text{if } y \in N(t), \\ 1/q & \text{otherwise,} \end{cases}$$

so that the return parameter p and in-out parameter q control how strongly the walk backtracks or moves outward. The walks are treated as sentences and used to train a skip-gram model (Church 2017) with negative sampling, which maximizes, for every vertex v and every context vertex c within a window of w positions,

$$\log \sigma(z_v^\top z'_c) + \sum_{k=1}^K \mathbb{E}_{n_k \sim P_n} \log \sigma(-z_v^\top z'_{n_k})$$

where z and z' are the input and context vectors and K negative samples are drawn per positive pair. The input vectors $z_v \in \mathbb{R}^d$ are the embeddings. The following parameters are used for all datasets: $r = 10$ walks per vertex, walk length $l = 80$, window size $w = 10$, $K = 5$ negative samples, learning rate

0.01, and 10 training epochs. The return parameter p , in-out parameter q , and dimension d are dataset-specific and were chosen by a preliminary search over $p, q \in \{0.25, 0.5, 1, 2, 4\}$ and several embedding dimensions, scored by reconstruction F1 (Table 1). No information is shared between partitions during training: the skip-gram model of partition p_i is initialized and optimized independently of every other partition’s model.

Table 1: Node2vec return parameter p , in-out parameter q , and embedding dimension d for each dataset.

Dataset	p	q	d
CITeseer	0.5	0.25	50
AstroPh	2	0.25	50
Cit-HepPh	4	0.25	100
AS-Oregon	0.5	2	128

Since a vertex can belong to several partitions (only under LFM), the final embedding of vertex u is the mean of its per-partition embeddings:

$$z_u = \frac{1}{|\mathcal{R}_u|} \sum_{p \in \mathcal{R}_u} z_u^{(p)},$$

where $\mathcal{R}_u$ is the set of partitions containing u . Under LPA, $|\mathcal{R}_u| = 1$ and z_u is simply the embedding from the single partition containing u . All decoders below operate on these merged embeddings, so the decoder itself is trained once, on vertices from all partitions, rather than once per partition.

Link-prediction data and decoders

A uniformly random 10% of the edges of G is removed and kept as the positive test set E_{test}^+ ; the rest form G_{train} , on which partitioning and embedding are performed. The negative test set E_{test}^- contains $|E_{\text{test}}^+|$ vertex pairs (u, v) , $u \neq v$, drawn uniformly at random from $V \times V$ subject to $(u, v) \notin E$, so the test set is balanced.

Positive training pairs are edges of G_{train} and negative training pairs are uniformly random vertex pairs that are not edges of G_{train} , with as many negatives as positives. The training edges are randomly split 80%/20% into training and validation sets. This procedure is repeated 10 times with independent random splits and negatives, and the decoder with the lowest final validation loss is kept.

Given the merged embeddings $u = z(a)$ and $v = z(b)$ of two vertices a, b , three decoders producing a link score are compared.

The embeddings are combined with the elementwise (Hadamard) product (Grover and Leskovec 2016) and passed through a learned linear layer without bias:

$$s(u, v) = w^\top (u \odot v) = \sum_{i=1}^d w_i u_i v_i,$$

with learned weights $w \in \mathbb{R}^d$. This is the simplest decoder considered and is equivalent to logistic regression on the Hadamard feature vector.

A learned matrix $W \in \mathbb{R}^{d \times d}$ captures interactions between all pairs of coordinates of the two embeddings:

$$s(u, v) = u^\top W v,$$

without bias and without any symmetry constraint on W ; the Hadamard decoder is the special case in which W is restricted to be diagonal.

For both neural decoders the link probability is $\hat{y} = \sigma(s(u, v))$ and the parameters are trained by minimizing the binary cross-entropy

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

with minibatch stochastic gradient descent (no momentum), batch size 32, learning rate 0.1, and 10 epochs.

Instead of a hand-designed interaction between the two embeddings, a random forest (Breiman 2001) is trained on the concatenated feature vector $[u; v] \in \mathbb{R}^{2d}$. The forest consists of 20 decision trees, each grown on a bootstrap sample of the training pairs and choosing every split from a random subset of the features, with a fixed random seed; the predicted link probability is the mean of the trees' class-probability estimates. Because it builds axis-aligned splits directly on the raw coordinates, the random forest does not assume that the two embeddings live in a common, smoothly varying latent geometry.

For all three decoders a pair is predicted to be an edge if $\hat{y} > 0.5$.

Benchmarks and evaluation metrics

The decoders are evaluated on the held-out pairs with two complementary protocols, alongside the reconstruction score used in (Yip et al. 2023).

Classification metrics: on the balanced test set $E_{\text{test}}^+ \cup E_{\text{test}}^-$, precision, recall, F1 score, and accuracy of the edge/non-edge decision at threshold 0.5 are computed. Accuracy is used in the preliminary comparison of partitioning strategies.

Ranking metrics: following the per-positive ranking design of the HeaRT benchmark (Li et al. 2023), but with uniformly random rather than heuristically

selected negatives, for each of up to 1000 randomly sampled positive test edges (a, b) , 500 negative candidates (a, c) are generated by drawing c uniformly at random from V such that (a, c) is not an edge of G_{train} . The true edge and its candidates are scored by the decoder, and the rank of the true edge is

$$\text{rank}(a, b) = 1 + |\{c : \hat{y}(a, c) > \hat{y}(a, b)\}|,$$

so that ties are resolved in favor of the true edge. Over the sampled set S of positive edges, the following are reported: **Hits@k**, the fraction of positive edges ranked within the top k , for $k \in \{1, 3, 10\}$; and the **Mean Reciprocal Rank**,

$$\text{MRR} = \frac{1}{|S|} \sum_{(a,b) \in S} \frac{1}{\text{rank}(a, b)};$$

Unlike the balanced classification metrics, the ranking metrics ask whether the true neighbor scores above hundreds of alternatives for the same source vertex, which is a considerably harder and more discriminative task.

RESTORE F1 (graph reconstruction): following (Yip et al. 2023), the graph is reconstructed from the merged embeddings by connecting the $m = |E_{\text{train}}|$ closest vertex pairs in the embedding space, and the F1 score of the neighborhood-level precision and recall with respect to G_{train} is computed.

Higher F1-scores indicate better preservation of graph structure in the embeddings. Let $G = (V, E)$ be the original graph and $G' = (V, E')$ be the reconstructed graph from embeddings. G' is constructed by connecting the m closest vertex pairs in the embedding space, where m is the number of edges in the original graph. The F1-score is calculated as follows:

$$F1 = 2 * \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

where

$$\begin{aligned} \text{Precision} &= \frac{1}{|V|} \sum_{v \in V} \frac{|N(v) \cap N'(v)|}{|N'(v)|} \\ \text{Recall} &= \frac{1}{|V|} \sum_{v \in V} \frac{|N(v) \cap N'(v)|}{|N(v)|} \end{aligned}$$

and $N(v)$ and $N'(v)$ are the neighbors of vertex v in the original and reconstructed graphs, respectively.

It is evaluated on the training graph, not on the held-out edges, and is reported alongside the link-prediction metrics specifically to contrast reconstruction quality against generalization quality on the same set of embeddings.

Results and discussion

Experimental setup

The main experiment covers four datasets (CITESEER, AstroPh, Cit-HepPh, AS-Oregon), four partition counts $P \in \{1, 2, 4, 8\}$, and the three decoders (bilinear, Hadamard, random forest), with 10 independent runs per configuration, i.e. $4 \times 4 \times 3 \times 10 = 480$ runs. All runs use LPA partitioning and the dataset-specific node2vec parameters of Table 1. A run repeats the whole pipeline from scratch: a new random hold-out split, a new partitioning, new random walks and skip-gram training, and new decoder training, so the 10 runs of a configuration are independent samples. Tables report the mean $\pm$ standard deviation over these 10 runs. Configurations are compared with the two-sided Mann-Whitney U test together with Cohen’s d (computed with the pooled standard deviation), at $\alpha = 0.05$ and without correction for multiple comparisons; with $n = 10$ per group, a large $|d|$ accompanies most significant differences, and the effect size rather than the p -value should carry the practical interpretation.

The RESTORE F1 score of a run is computed from the merged embeddings before any decoder is trained, so it does not depend on the decoder. This serves as a sanity check of the pipeline: when the F1 scores of two decoder sweeps are compared, only 2 of 16 (dataset, P) groups differ significantly for the bilinear-vs-Hadamard comparison, 0 of 16 for bilinear-vs-random-forest, and 1 of 16 for Hadamard-vs-random-forest, with no consistent sign, which is the run-to-run noise expected from independent partitionings.

Datasets

Experiments are performed on four real-world graph datasets, summarized in the following table.

Table 2: Datasets used in the experiments.

Dataset	Nodes	Edges	Average degree	Average cluster-coefficient	Density	Modularity
CITESEER	3264	4536	2.78	0.145	0.0009	0.72
AstroPh	18772	198110	21.11	0.631	0.00112	0.33
AS-Oregon	11806	38781	6.57	0.399	0.00056	0.57
Cit-HepPh	34546	420921	24.37	0.285	0.00071	0.67

CITESEER is a citation network of scientific publications. AstroPh is a collabo-

ration/citation network derived from SNAP, where AstroPh links co-authors of astrophysics papers. AS-Oregon is a network of autonomous systems and their peering connections. Cit-HepPh is a citation network of high-energy physics papers. These datasets vary in size and density, which allows the effect of partitioning to be observed under different community structures.

Impact of partitioning strategy

Table 3 compares LFM and LPA on CITESEER across partition counts, together with the single-partition baseline. This is a preliminary, single-dataset experiment run with an earlier version of the evaluation protocol, so its absolute numbers are not comparable to the main experiment below; it is reported because it motivates the use of LPA throughout. “LP accuracy” is the accuracy of the edge/non-edge decision at threshold 0.5 on the held-out pairs.

Table 3: CITESEER link-prediction accuracy and RESTORE F1 for LFM and LPA partitioning across partition counts (preliminary experiment).

Strategy	Partitions	LP Accuracy	RESTORE F1
Baseline	1	0.9209 ± 0.0105	0.3782 ± 0.0110
LFM	2	0.6884 ± 0.0109	0.3154 ± 0.0122
LFM	4	0.5576 ± 0.0159	0.2693 ± 0.0087
LFM	8	0.5401 ± 0.0173	0.2099 ± 0.0065
LPA	2	0.8026 ± 0.0193	0.4516 ± 0.0360
LPA	4	0.7485 ± 0.0061	0.4816 ± 0.0128
LPA	8	0.7128 ± 0.0200	0.5401 ± 0.0107
LPA	16	0.6874 ± 0.0148	0.6264 ± 0.0133
LPA	32	0.6765 ± 0.0169	0.7017 ± 0.0049
LPA	64	0.6681 ± 0.0114	0.6922 ± 0.0047
LPA	128	0.6888 ± 0.0180	0.6569 ± 0.0141
LPA	256	0.6791 ± 0.0157	0.6293 ± 0.0199

At every partition count where both strategies were measured (2, 4, 8), LPA gives substantially higher link-prediction accuracy than LFM (0.8026 vs. 0.6884 at $P = 2$, 0.7128 vs. 0.5401 at $P = 8$) and also higher RESTORE F1. A plausible reason is that LPA yields disjoint communities that are only grouped, never split, by the bin-packing step, whereas LFM’s overlapping communities and randomly assigned uncovered vertices give partitions that cut more of the local neighborhood structure. Since this evidence comes from one dataset, the preference for LPA is a working choice rather than an established general result, and all remaining experiments use LPA.

Table 3 also shows, for LPA, that accuracy falls from 0.9209 at $P = 1$ to 0.6681 at $P = 64$ while RESTORE F1 rises from 0.3782 to a peak of 0.7017 at $P = 32$;

the same divergence is examined systematically in the main experiment below.

Effect of the partition count on link prediction

Tables 4–7 report the ranking metrics of the three decoders for every dataset and partition count. Hits@ k is the fraction of held-out positive edges ranked within the top k among 500 negative candidates, and MRR is the mean reciprocal rank (Methods).

Table 4: Mean reciprocal rank (MRR), in percent, mean $\pm$ standard deviation over 10 runs, LPA partitioning.

Dataset	Decoder	P=1	P=2	P=4	P=8
CITeseer	Bilinear	27.61 $\pm$ 1.82	17.90 $\pm$ 1.80	13.80 $\pm$ 0.71	10.94 $\pm$ 1.17
CITeseer	Hadamard	22.92 $\pm$ 2.80	13.88 $\pm$ 1.88	12.49 $\pm$ 1.37	13.87 $\pm$ 1.66
CITeseer	RDF	32.27 $\pm$ 1.47	28.02 $\pm$ 2.03	24.75 $\pm$ 1.77	23.99 $\pm$ 1.37
AstroPh	Bilinear	68.74 $\pm$ 1.32	51.03 $\pm$ 1.55	45.19 $\pm$ 2.19	39.64 $\pm$ 2.28
AstroPh	Hadamard	32.34 $\pm$ 4.13	26.84 $\pm$ 3.22	28.24 $\pm$ 1.63	29.63 $\pm$ 2.50
AstroPh	RDF	62.73 $\pm$ 1.28	62.05 $\pm$ 1.87	62.70 $\pm$ 1.40	61.95 $\pm$ 0.91
Cit-HepPh	Bilinear	62.95 $\pm$ 1.13	44.85 $\pm$ 3.05	39.60 $\pm$ 2.81	36.91 $\pm$ 2.02
Cit-HepPh	Hadamard	37.05 $\pm$ 2.79	28.38 $\pm$ 1.22	27.15 $\pm$ 1.43	32.15 $\pm$ 1.98
Cit-HepPh	RDF	53.23 $\pm$ 0.97	49.86 $\pm$ 1.59	47.96 $\pm$ 0.75	47.55 $\pm$ 1.08
AS-Oregon	Bilinear	25.08 $\pm$ 0.95	18.69 $\pm$ 1.05	15.96 $\pm$ 1.51	15.34 $\pm$ 1.63
AS-Oregon	Hadamard	14.48 $\pm$ 1.68	10.47 $\pm$ 0.78	9.12 $\pm$ 1.08	9.97 $\pm$ 0.86
AS-Oregon	RDF	43.26 $\pm$ 1.49	46.25 $\pm$ 1.68	46.09 $\pm$ 1.69	45.13 $\pm$ 1.13

Table 5: Hits@1, in percent, mean $\pm$ standard deviation over 10 runs, LPA partitioning.

Dataset	Decoder	P=1	P=2	P=4	P=8
CITeseer	Bilinear	17.70 $\pm$ 1.14	11.08 $\pm$ 1.79	8.08 $\pm$ 0.99	5.76 $\pm$ 1.10
CITeseer	Hadamard	14.37 $\pm$ 2.65	8.21 $\pm$ 1.77	7.64 $\pm$ 1.59	9.38 $\pm$ 1.50
CITeseer	RDF	23.91 $\pm$ 1.43	21.55 $\pm$ 1.96	18.79 $\pm$ 1.72	18.28 $\pm$ 1.61
AstroPh	Bilinear	56.79 $\pm$ 1.58	38.54 $\pm$ 1.70	32.57 $\pm$ 2.29	27.68 $\pm$ 2.34
AstroPh	Hadamard	20.73 $\pm$ 3.46	16.93 $\pm$ 2.98	18.53 $\pm$ 1.79	19.56 $\pm$ 2.49
AstroPh	RDF	54.44 $\pm$ 1.42	53.86 $\pm$ 1.85	54.79 $\pm$ 1.53	53.81 $\pm$ 1.38
Cit-HepPh	Bilinear	48.38 $\pm$ 1.47	31.05 $\pm$ 2.81	27.10 $\pm$ 2.54	24.90 $\pm$ 1.91
Cit-HepPh	Hadamard	22.45 $\pm$ 2.49	17.08 $\pm$ 1.24	16.08 $\pm$ 1.42	19.93 $\pm$ 1.69
Cit-HepPh	RDF	40.31 $\pm$ 0.92	37.57 $\pm$ 1.81	35.82 $\pm$ 1.05	35.31 $\pm$ 1.32
AS-Oregon	Bilinear	14.99 $\pm$ 1.07	9.70 $\pm$ 1.23	7.83 $\pm$ 1.13	7.57 $\pm$ 1.40
AS-Oregon	Hadamard	6.99 $\pm$ 1.44	4.92 $\pm$ 0.73	4.08 $\pm$ 0.88	4.49 $\pm$ 0.91
AS-Oregon	RDF	38.06 $\pm$ 1.60	41.44 $\pm$ 1.77	41.58 $\pm$ 1.75	40.47 $\pm$ 1.08

Table 6: Hits@3, in percent, mean $\pm$ standard deviation over 10 runs, LPA partitioning.

Dataset	Decoder	P=1	P=2	P=4	P=8
CITeseER	Bilinear	32.25 $\pm$ 2.93	19.69 $\pm$ 1.80	15.54 $\pm$ 1.02	12.01 $\pm$ 1.69
CITeseER	Hadamard	25.94 $\pm$ 3.57	15.56 $\pm$ 2.43	13.75 $\pm$ 1.36	15.67 $\pm$ 1.49
CITeseER	RDF	35.70 $\pm$ 2.07	30.40 $\pm$ 2.22	26.91 $\pm$ 2.05	26.14 $\pm$ 1.59
AstroPh	Bilinear	77.48 $\pm$ 1.50	58.30 $\pm$ 2.02	51.70 $\pm$ 2.66	44.71 $\pm$ 3.24
AstroPh	Hadamard	36.65 $\pm$ 4.97	29.82 $\pm$ 3.68	30.90 $\pm$ 1.76	32.72 $\pm$ 2.94
AstroPh	RDF	66.61 $\pm$ 1.61	66.12 $\pm$ 2.10	66.49 $\pm$ 1.64	65.79 $\pm$ 0.88
Cit-HepPh	Bilinear	73.21 $\pm$ 1.82	51.77 $\pm$ 4.09	45.33 $\pm$ 4.16	42.48 $\pm$ 2.77
Cit-HepPh	Hadamard	43.07 $\pm$ 3.61	32.01 $\pm$ 1.33	30.70 $\pm$ 1.90	37.74 $\pm$ 2.51
Cit-HepPh	RDF	59.91 $\pm$ 1.30	55.79 $\pm$ 1.55	53.65 $\pm$ 1.59	53.51 $\pm$ 1.42
AS-Oregon	Bilinear	29.22 $\pm$ 1.24	21.21 $\pm$ 1.34	17.64 $\pm$ 1.63	16.60 $\pm$ 2.03
AS-Oregon	Hadamard	15.45 $\pm$ 2.20	10.43 $\pm$ 1.09	9.12 $\pm$ 1.22	10.19 $\pm$ 1.17
AS-Oregon	RDF	44.35 $\pm$ 1.48	47.14 $\pm$ 1.69	46.99 $\pm$ 1.70	46.25 $\pm$ 1.53

Table 7: Hits@10, in percent, mean $\pm$ standard deviation over 10 runs, LPA partitioning.

Dataset	Decoder	P=1	P=2	P=4	P=8
CITeseER	Bilinear	47.37 $\pm$ 3.50	30.71 $\pm$ 2.55	24.50 $\pm$ 1.34	20.73 $\pm$ 1.73
CITeseER	Hadamard	38.96 $\pm$ 3.50	24.46 $\pm$ 2.45	21.52 $\pm$ 1.72	21.39 $\pm$ 2.55
CITeseER	RDF	48.23 $\pm$ 2.61	39.67 $\pm$ 2.87	35.59 $\pm$ 2.35	34.02 $\pm$ 2.01
AstroPh	Bilinear	89.91 $\pm$ 0.78	74.46 $\pm$ 1.37	69.70 $\pm$ 1.91	63.97 $\pm$ 2.55
AstroPh	Hadamard	55.58 $\pm$ 5.68	46.86 $\pm$ 4.04	47.57 $\pm$ 2.19	49.85 $\pm$ 2.50
AstroPh	RDF	78.96 $\pm$ 1.45	77.90 $\pm$ 2.41	77.85 $\pm$ 1.23	77.67 $\pm$ 0.97
Cit-HepPh	Bilinear	89.85 $\pm$ 0.56	72.40 $\pm$ 3.12	64.52 $\pm$ 2.83	60.36 $\pm$ 2.20
Cit-HepPh	Hadamard	67.77 $\pm$ 3.84	52.46 $\pm$ 1.58	50.80 $\pm$ 1.67	57.23 $\pm$ 2.52
Cit-HepPh	RDF	79.78 $\pm$ 1.24	74.61 $\pm$ 1.66	72.98 $\pm$ 1.27	72.38 $\pm$ 1.37
AS-Oregon	Bilinear	45.39 $\pm$ 1.44	36.97 $\pm$ 1.19	32.74 $\pm$ 3.00	31.59 $\pm$ 2.28
AS-Oregon	Hadamard	29.62 $\pm$ 2.72	21.46 $\pm$ 1.93	18.50 $\pm$ 2.07	20.43 $\pm$ 1.33
AS-Oregon	RDF	53.71 $\pm$ 1.27	55.58 $\pm$ 1.82	54.53 $\pm$ 1.55	53.72 $\pm$ 1.45

The three decoders respond to partitioning very differently, and the pattern is the same for all four ranking metrics.

Bilinear. Every metric decreases monotonically with P on every dataset, and the largest single drop is always between $P = 1$ and $P = 2$: for example, AstroPh MRR falls from 68.74% to 51.03% and Hits@1 from 56.79% to 38.54%, and Cit-HepPh MRR from 62.95% to 44.85%. The decline continues, more slowly, up to $P = 8$.

Hadamard. The drop from $P = 1$ to $P = 2$ is also clear on all datasets, but for larger P the curve flattens and on several datasets partially recovers: for example, Cit-HepPh MRR is 28.38% at $P = 2$, 27.15% at $P = 4$, and 32.15% at $P = 8$, and CITESEER MRR is 12.49% at $P = 4$ and 13.87% at $P = 8$. The Hadamard decoder is also much weaker than the bilinear one even without partitioning (AstroPh MRR at $P = 1$: 32.34% vs. 68.74%), so part of its apparent robustness to partitioning is simply that it has less quality to lose.

Random forest. The random forest is the least affected decoder, but the effect depends on the dataset. On AS-Oregon and AstroPh the metrics are essentially flat in P : AS-Oregon MRR is 43.26% at $P = 1$, 46.25% at $P = 2$, and 45.13% at $P = 8$, and AstroPh MRR moves only from 62.73% to 61.95%. On CITESEER and Cit-HepPh the ranking quality declines, from 32.27% to 23.99% and from 53.23% to 47.55% MRR respectively, and the decline happens in the first two doublings of P , after which the metrics change little.

Table 8: Relative change of the mean MRR from $P = 1$ to $P = 8$, per decoder.

Dataset	Bilinear	Hadamard	RDF
CITESEER	-60%	-39%	-26%
AstroPh	-42%	-8%	-1%
Cit-HepPh	-41%	-13%	-11%
AS-Oregon	-39%	-31%	+4%

Table 8 summarizes the effect as the relative change of the mean MRR between $P = 1$ and $P = 8$: the bilinear decoder loses 39–60% of its MRR, the Hadamard decoder 8–39%, and the random forest between +4% and −26%.

Significance of the partition-count effect

Table 9 tests the effect of each doubling of P on the MRR, for every decoder, using Cohen’s d (negative means MRR decreases with more partitions) and marking with an asterisk the steps that are significant under the Mann–Whitney test.

Table 9: Cohen’s d of the MRR between adjacent partition counts ($n = 10$ runs per group); * marks Mann–Whitney $p < 0.05$.

Dataset	Step	Bilinear	Hadamard	RDF
CITESEER	$P: 1 \rightarrow 2$	-5.35*	-3.79*	-2.40*
CITESEER	$P: 2 \rightarrow 4$	-2.98*	-0.84	-1.72*
CITESEER	$P: 4 \rightarrow 8$	-2.96*	+0.91*	-0.48
AstroPh	$P: 1 \rightarrow 2$	-12.27*	-1.49*	-0.43
AstroPh	$P: 2 \rightarrow 4$	-3.07*	+0.55	+0.40

Dataset	Step	Bilinear	Hadamard	RDF
AstroPh	$P: 4 \rightarrow 8$	-2.48*	+0.66	-0.64
Cit-HepPh	$P: 1 \rightarrow 2$	-7.86*	-4.03*	-2.55*
Cit-HepPh	$P: 2 \rightarrow 4$	-1.79*	-0.93	-1.52*
Cit-HepPh	$P: 4 \rightarrow 8$	-1.10*	+2.90*	-0.44
AS-Oregon	$P: 1 \rightarrow 2$	-6.37*	-3.06*	+1.88*
AS-Oregon	$P: 2 \rightarrow 4$	-2.10*	-1.43*	-0.09
AS-Oregon	$P: 4 \rightarrow 8$	-0.39	+0.86	-0.67

For the bilinear decoder, 11 of the 12 steps are significant and all effects are negative, with very large effect sizes for the first step ($|d|$ between 5.4 and 12.3); the only non-significant step is AS-Oregon $P = 4 \rightarrow 8$ ($p = 0.43$), where the MRR has essentially plateaued. For the Hadamard decoder, 7 of 12 steps are significant; the $P = 1 \rightarrow 2$ step is significantly negative on all datasets, but the later steps are mostly not significant, and two of them are significantly *positive* (CITeseer and Cit-HepPh, $P = 4 \rightarrow 8$), i.e. the MRR recovers. For the random forest, 5 of 12 steps are significant. The significant steps are the first two doublings on CITeseer ($d = -2.40$ and -1.72) and on Cit-HepPh ($d = -2.55$ and -1.52), plus a significantly *positive* $P = 1 \rightarrow 2$ step on AS-Oregon ($d = +1.88$); no step is significant on AstroPh, and no $P = 4 \rightarrow 8$ step is significant on any dataset, i.e. the MRR plateaus. Partitioning therefore has a large, monotone, and highly significant effect on the bilinear decoder, a mostly one-time effect on the Hadamard decoder, and a dataset-dependent, plateauing effect on the random forest, which is moderate on CITeseer and Cit-HepPh and absent or slightly positive on AstroPh and AS-Oregon.

Comparison of decoders

Table 10 compares the decoders directly on the MRR in every (dataset, P) group, again with Cohen’s d and Mann–Whitney significance (asterisk); a positive value means the first-named decoder scores higher.

Table 10: Cohen’s d of the MRR between decoders in each (dataset, P) group ($n = 10$); * marks Mann–Whitney $p < 0.05$; positive means the first-named decoder is higher.

Dataset	P	Bilinear – Hadamard	RDF – Bilinear	RDF – Hadamard
CITeseer	1	+1.99*	+2.81*	+4.18*
CITeseer	2	+2.18*	+5.28*	+7.23*
CITeseer	4	+1.20	+8.11*	+7.75*
CITeseer	8	-2.05*	+10.26*	+6.66*
AstroPh	1	+11.87*	-4.62*	+9.95*
AstroPh	2	+9.57*	+6.42*	+13.38*

Dataset	P	Bilinear – Hadamard	RDF – Bilinear	RDF – Hadamard
AstroPh	4	+8.76*	+9.52*	+22.67*
AstroPh	8	+4.18*	+12.83*	+17.19*
Cit-HepPh	1	+12.17*	-9.24*	+7.75*
Cit-HepPh	2	+7.08*	+2.06*	+15.12*
Cit-HepPh	4	+5.59*	+4.07*	+18.21*
Cit-HepPh	8	+2.38*	+6.57*	+9.67*
AS-Oregon	1	+7.77*	+14.52*	+18.14*
AS-Oregon	2	+8.86*	+19.63*	+27.23*
AS-Oregon	4	+5.21*	+18.78*	+26.02*
AS-Oregon	8	+4.12*	+21.24*	+35.08*

The decoders are ordered **random forest** > **bilinear** > **Hadamard** in most configurations. The bilinear decoder is significantly better than the Hadamard decoder in 14 of 16 groups, usually with very large effect sizes, the exceptions being CITESEER $P = 4$ (a tie) and CITESEER $P = 8$, where the Hadamard decoder is significantly better (the same holds for Hits@1 and Hits@3). The random forest is significantly better than the Hadamard decoder in all 16 groups on all four ranking metrics (d from +3.0 to +36.0) and significantly better than the bilinear decoder on MRR in 14 of 16 groups (d up to +21.2). Its advantage over the bilinear decoder grows with the partition count, because the bilinear decoder degrades faster than the random forest (on AS-Oregon, d grows from +14.5 at $P = 1$ to +21.2 at $P = 8$). The only exceptions are the unpartitioned dense graphs: at $P = 1$ on AstroPh and Cit-HepPh the bilinear decoder is significantly better than the random forest (MRR 68.74% vs. 62.73% and 62.95% vs. 53.23%).

Reconstruction quality versus link prediction

Table 11: RESTORE F1 of the reconstruction from the merged embeddings, in percent, mean $\pm$ standard deviation over 10 runs (values of the bilinear sweep; the other two sweeps agree within run-to-run noise since the score does not depend on the decoder).

Dataset	P=1	P=2	P=4	P=8
CITESEER	35.88 $\pm$ 1.46	43.92 $\pm$ 3.25	49.72 $\pm$ 2.34	55.68 $\pm$ 6.12
AstroPh	70.62 $\pm$ 0.08	67.76 $\pm$ 0.09	67.84 $\pm$ 0.12	67.75 $\pm$ 0.11
Cit-HepPh	54.87 $\pm$ 0.06	57.41 $\pm$ 0.79	60.35 $\pm$ 2.12	61.59 $\pm$ 0.87
AS-Oregon	33.88 $\pm$ 0.28	38.88 $\pm$ 1.91	45.97 $\pm$ 3.79	43.20 $\pm$ 1.78

Table 11 shows the RESTORE F1 of the embeddings used in this experiment: it *increases* with P on CITESEER (35.88% $\rightarrow$ 55.68%), Cit-HepPh (54.87% $\rightarrow$

61.59%), and AS-Oregon (33.88% $\rightarrow$ 43.20%), and is essentially flat on AstroPh apart from a small drop of about three points between $P = 1$ and $P = 2$. Compared with Tables 4–7, the reconstruction score and the neural-decoder link-prediction quality therefore move in *opposite directions* as the partition count grows: on CITESEER, for instance, F1 rises by more than 19 points while the bilinear MRR falls by 60%. Partitioning makes each partition’s local reconstruction task easier and more self-contained, which the reconstruction score rewards, whereas link prediction on held-out edges additionally requires comparing embeddings of vertices that may lie in different partitions and were trained independently.

The pooled Spearman correlation between the per-run F1 and MRR is positive and significant for all three decoders (bilinear $\rho = 0.62$, Hadamard $\rho = 0.65$, RDF $\rho = 0.72$, $n = 160$ each, $p < 10^{-17}$), but this does not contradict the opposite trends above: it is dominated by differences between datasets (graphs that are easy to embed score high on both), while, within a dataset, the partition count pushes the two quantities apart. Reconstruction F1 is therefore a reasonable indicator of how learnable a graph is, but not a reliable proxy for how well partitioned embeddings support link prediction.

Discussion

The main results are consistent with a single explanation. Because every partition is embedded independently, the embedding spaces of different partitions are unrelated, so a pair of vertices from different partitions is scored from vectors that are not comparable. The neural decoders learn one smooth scoring function ($w^\top(u \odot v)$ or $u^\top Wv$) that presupposes a shared latent geometry, and the first partition split, which introduces the first cross-partition pairs, is where they lose most (Tables 4–7, Table 9). The random forest builds axis-aligned splits directly on the raw coordinates of $[u; v]$ and does not assume such a geometry, and it is the decoder least affected by partitioning, although it too loses ranking quality on the two datasets (CITESEER, Cit-HepPh) where the first partition splits cut the most structure. We stress that this explanation is consistent with the evidence but was not tested directly: the experiments do not separate within-partition test pairs from cross-partition ones.

The practical recommendations are as follows. When the graph must be partitioned, use LPA and a random forest on the concatenated embeddings, which gives the best ranking quality in every configuration with $P \geq 2$ and degrades much less with the number of partitions than the neural decoders. When the graph fits in a single partition and is dense (AstroPh, Cit-HepPh), the bilinear decoder is the best choice. The Hadamard decoder is dominated by the other two in almost every configuration and is not recommended.

Limitations. (i) With 20 trees the forest’s probabilities take only 21 distinct values, and ties in the ranking are resolved in favor of the true edge, which can favor the random forest relative to the neural decoders. (ii) Neither the neural

decoders nor the random forest were tuned separately for each partition count. (iii) The comparison of LFM and LPA is limited to one dataset and an earlier evaluation protocol, and all main experiments use LPA and four datasets. (iv) The significance tests are not corrected for multiple comparisons.

Conclusion

This paper studied how graph partitioning and the choice of link-prediction decoder affect link prediction over node embeddings that are computed independently on each partition with static node2vec. On four datasets, four partition counts, and 10 runs per configuration, we compared a bilinear decoder, a Hadamard decoder, and a random forest on concatenated embeddings using Hits@ k and MRR, with Mann–Whitney tests and Cohen’s d .

Partitioning strongly degrades the neural decoders: from $P = 1$ to $P = 8$ the MRR of the bilinear decoder decreases by 39–60% (significantly at 11 of 12 steps) and that of the Hadamard decoder by 8–39%, mostly in the first step. The random forest is far less sensitive: its MRR changes between +4% and –26%, with 5 of 12 steps significant, the decline being confined to CITESEER and Cit-HepPh and to the first two doublings of P . Consequently, the decoders are ordered random forest > bilinear > Hadamard in most configurations, with the bilinear decoder best only on unpartitioned dense graphs. In a preliminary single-dataset comparison, LPA gave clearly better link prediction than LFM at every partition count tested. Finally, the reconstruction F1 of the same embeddings *increases* with the partition count on three of the four datasets while neural-decoder link-prediction quality decreases, so reconstruction quality is not a reliable proxy for downstream link-prediction quality in a partitioned embedding pipeline, a distinction that Seminar 3’s reconstruction-only study could not surface.

Future work should (a) repeat the ranking evaluation with random tie-breaking, (b) split the test pairs into within-partition and cross-partition pairs to test directly whether cross-partition pairs cause the degradation, (c) tune the neural decoders per partition count and consider aligning the per-partition embedding spaces (for instance by a learned linear map) before decoding, (d) extend the LFM/LPA comparison and the decoder study to more datasets, and (e) study the replication factor, which lets a vertex be embedded in several partitions, as a means to recover link-prediction quality.

References

- Adamic, Lada A, and Natalie Glance. 2005. “The Political Blogosphere and the 2004 US Election: Divided They Blog.” *Proceedings of the 3rd International Workshop on Link Discovery*, 36–43.

- Backstrom, Lars, Dan Huttenlocher, Jon Kleinberg, and Xiangyang Lan. 2006. “Group Formation in Large Social Networks: Membership, Growth, and Evolution.” *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 44–54.
- Benlic, Una, and Jin-Kao Hao. 2010. “An Effective Multilevel Memetic Algorithm for Balanced Graph Partitioning.” *2010 22nd IEEE International Conference on Tools with Artificial Intelligence* 1: 121–28.
- Bordes, Antoine, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. 2013. “Translating Embeddings for Modeling Multi-Relational Data.” *Advances in Neural Information Processing Systems* 26.
- Breiman, Leo. 2001. “Random Forests.” *Machine Learning* 45 (1): 5–32.
- Çatalyürek, Ümit, Karen Devine, Marcelo Faraj, et al. 2023. “More Recent Advances in (Hyper) Graph Partitioning.” *ACM Computing Surveys* 55 (12): 1–38.
- Church, Kenneth Ward. 2017. “Word2Vec.” *Natural Language Engineering* 23 (1): 155–62.
- Fang, Peng, Arijit Khan, Siqiang Luo, et al. 2023. “Distributed Graph Embedding with Information-Oriented Random Walks.” *arXiv Preprint arXiv:2303.15702*.
- Grover, Aditya, and Jure Leskovec. 2016. “Node2vec: Scalable Feature Learning for Networks.” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 855–64.
- Gupta, Jatinder ND, and Johnny C Ho. 1999. “A New Heuristic Algorithm for the One-Dimensional Bin-Packing Problem.” *Production Planning & Control* 10 (6): 598–603.
- Krebs, Valdis E. 2002. “Mapping Networks of Terrorist Cells.” *Connections* 24 (3): 43–52.
- Lancichinetti, Andrea, Santo Fortunato, and János Kertész. 2009. “Detecting the Overlapping and Hierarchical Community Structure in Complex Networks.” *New Journal of Physics* 11 (3): 033015.
- Leskovec, Jure, Daniel Huttenlocher, and Jon Kleinberg. 2010a. “Predicting Positive and Negative Links in Online Social Networks.” *Proceedings of the 19th International Conference on World Wide Web*, 641–50.
- Leskovec, Jure, Daniel Huttenlocher, and Jon Kleinberg. 2010b. “Signed

- Networks in Social Media.” *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 1361–70.
- Li, Juanhui, Harry Shomer, Haitao Mao, et al. 2023. “Evaluating Graph Neural Networks for Link Prediction: Current Pitfalls and New Benchmarking.” *Advances in Neural Information Processing Systems 36 (Datasets and Benchmarks Track)*.
- Lombardo, Gianfranco, and Agostino Poggi. 2019. “A Scalable and Distributed Actor-Based Version of the Node2Vec Algorithm.” *Proceedings of the 20th Workshop From Objects to Agents* (Parma, Italy), 134–41.
- Raghavan, Usha Nandini, Réka Albert, and Soundar Kumara. 2007. “Near Linear Time Algorithm to Detect Community Structures in Large-Scale Networks.” *Physical Review E* 76 (3). <https://doi.org/10.1103/physreve.76.036106>.
- Romero Ruiz, Emmanuel, and Carlos Segura. 2018. “Memetic Algorithm with Hungarian Matching Based Crossover and Diversity Preservation.” *Computación y Sistemas* 22 (2): 347–61.
- Rozemberczki, Benedek, and Rik Sarkar. 2021. *Twitch Gamers: A Dataset for Evaluating Proximity Preserving and Structural Role-Based Node Embeddings*.
- Sanders, Peter, and Christian Schulz. 2011. “Engineering Multilevel Graph Partitioning Algorithms.” *European Symposium on Algorithms*, 469–80.
- Sanders, Peter, and Christian Schulz. 2012. “Distributed Evolutionary Graph Partitioning.” *2012 Proceedings of the Fourteenth Workshop on Algorithm Engineering and Experiments (ALENEX)*, 16–29.
- Savić, Miloš, Mirjana Ivanović, and Bojana Dimić Surla. 2017. “Analysis of Intra-Institutional Research Collaboration: A Case of a Serbian Faculty of Sciences.” *Scientometrics* 110: 195–216.
- Van Der Hofstad, Remco. 2024. *Random Graphs and Complex Networks*. Vol. 54. Cambridge university press.
- Yip, Hong Yung, Chidaksh Ravuru, Neelabha Banerjee, et al. 2023. *RESTORE: Graph Embedding Assessment Through Reconstruction*. <https://arxiv.org/abs/2308.14659>.